

Maven

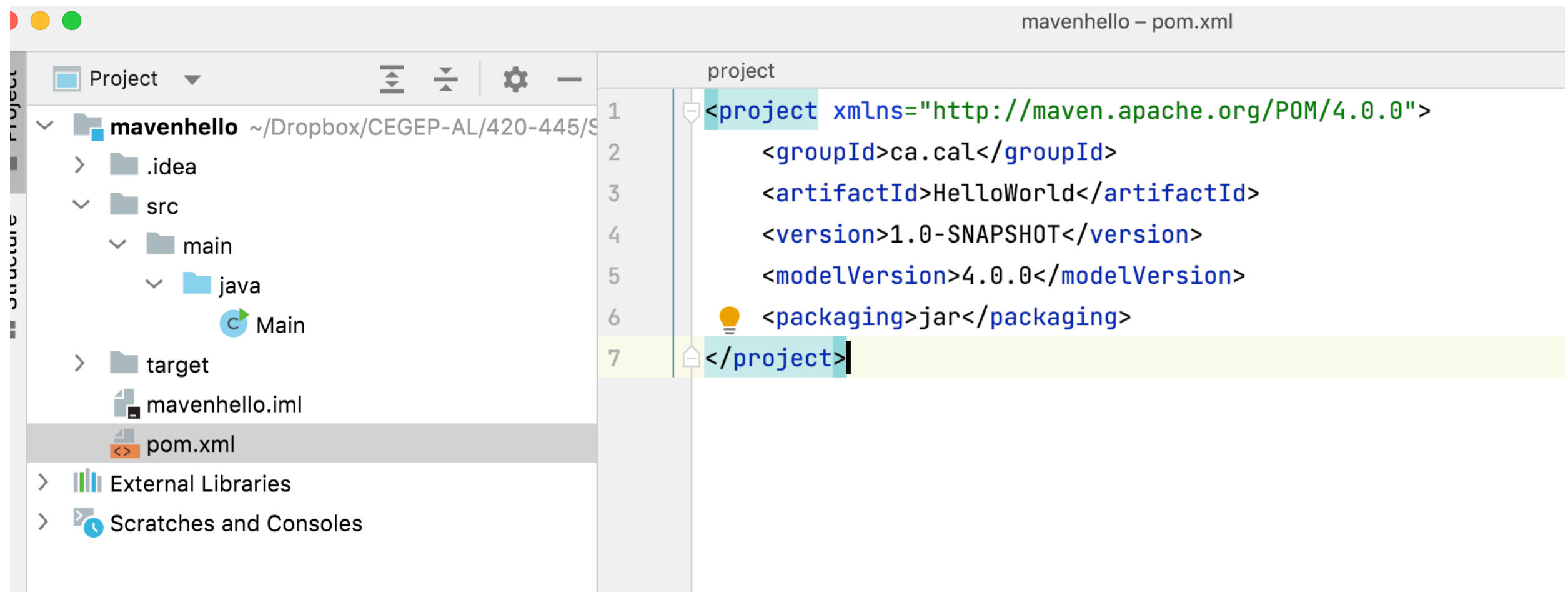
C'est quoi ?

- Build tool
 - 1 artéfact
 - Gère les dépendances (et dépendances transitives)
- Gestion de projet
 - Numéro de version et ne produit qu'une seule version d'un artéfact. Gère les 'releases'
 - Décrit le projet
 - Produit la javadocs

Pourquoi utiliser Maven ?

- Produit des 'builds' répétables
- Le code est indépendant de la plate-forme sur lequel il est bati ou compilé. Ces infos sont 'externalisées' par rapport à l'application.
- Gère les dépendances transitives
- Environnement: Contient toute l'information pour faire un 'build' du code
- Local Repo (~/.m2)
- IDE ou standalone
- Méthode préférée pour utiliser dans les systèmes d'intégration continue. (Jenkins, ...)
- DEMO HelloWorld (Semaine 1/mavenhello)

Maven HelloWorld



- À la ligne de commande, faire 'mvn clean'
- 'mvn compile'

Maven HelloWorld

- mvn compile va produire une erreur. Il faut ajouter ce snippet pour que ça compile correctement.

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <groupId>ca.cal</groupId>
  <artifactId>HelloWorld</artifactId>
  <version>1.0-SNAPSHOT</version>
  <modelVersion>4.0.0</modelVersion>
  <packaging>jar</packaging>
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <release>17</release>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Maven

- `src/main/java` - Emplacement de notre code java
- Emplacement de nos 'packages' (Ne jamais utiliser le 'default' package de java)
- `src/test/java` - Emplacement pour l'écriture des tests
- Regardez comment le 'pom.xml' et la structure de répertoire sont liés.

Maven-dependencies

- Par convention de nom
- groupId, artifactId, version
- Maven va chercher les dépendances transitives pour nous.
- Exemple avec 'commons-lang'

Maven - dependencies

- Ajout de 'commons-lang3'

```
<dependencies>  
  <dependency>  
    <groupId>org.apache.commons</groupId>  
    <artifactId>commons-lang3</artifactId>  
    <version>3.12.0</version>  
  </dependency>  
</dependencies>
```

Maven - goals

- clean - détruit le 'target' directory
- compile - produit les classes dans 'target'
- test - roule tous les tests et arrête dans la chaînes de 'goal' s'il y a une erreur (ne fera pas 'package')
- package - produit 1 artéfact (ex: un jar)
- install - exécute 'package' et installe l'artéfact dans le répo local. Ce nouvel artéfact peut ensuite être utilisé dans d'autres projets à l'aide du mécanisme de dépendance
- deploy - exécute 'install' et déploie dans un répo remote.

Maven - IDE

- Convertir le projet en projet Maven
- Right-click sur le projet et choisir 'Add Framework Support...'
- Choisir Maven dans le choix proposé.
- Voir le nouveau menu à droite et les dépendances dans le projet qui ont été ajoutés.

Maven - dépendances...

- groupId, artifactId, versionId.
- Ce sont les éléments nécessaires à l'ajout des dépendances
- Attention à l'élément 'scope'. S'il est à 'test', il peut vous donner des surprises (ex: h2)

Maven - versions

- SNAPSHOT - permet de réutiliser la dernière version produite automatiquement. Le 'bleeding edge'. À noter que SNAPSHOT doit être en majuscule.
 - Ex: monapp-1.0-SNAPSHOT
 - À chaque fois que l'on 'build', maven va vérifier s'il y a une nouvelle version de snapshot disponible et l'utiliser si c'est le cas.
- RELEASE - Ex: monapp-1.0, ou monap-1.0.1.
 - À noter que RELEASE n'apparaît pas dans le nom, c'est libre de mettre ce qu'on veut. Ça permet de geler la version de code à utiliser. Très pratique en production.

Maven - versions

- Termes de l'industrie
 - monapp-1.0-M1.jar (Milestone 1)
 - monapp-1.0-RC1.jar (Release Candidate 1)
 - monapp-1.0-RELEASE.jar (version stable)
 - monapp-1.0-Final.jar (comme release, version stable)

Maven - packaging type

- pom, jar, war, ear, maven-plugin
- Default 'jar'

Dépendances transitives

- modifier le pom avec 'hibernate-core'
- Probablement la raison la plus importante pour l'utilisation de Maven

Maven - scopes

- compile - dépendance est disponible dans toutes les phases (compile, test, package, runtime)
- provided - ex:servlet api. Il ne sera pas 'packagé'
- runtime - pas disponible à la compilation (ex: jdbc.jar)
- test - disponible seulement durant les phases test, compilation et exécution. Pas inclus dans l'artéfact final.